5. Select the forecasting model. You'll notice that we have two versions of the model. Add an alias to each: **champion** and **challenger** are common to indicate the current best model and the newer version being considered to replace the current best model.

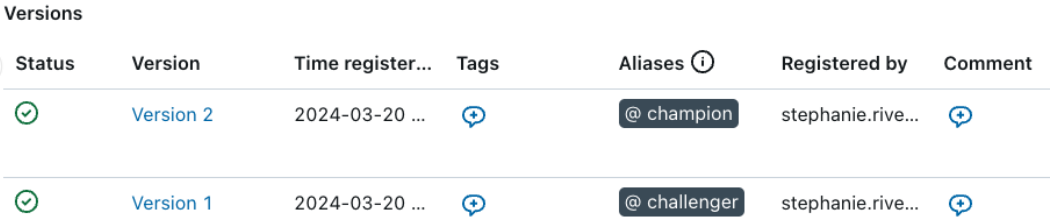


Figure 7.11 – Assign aliases to identify model status

Now it's time to think about how we want to deploy this model. Since this is a sales forecasting use case predicting 10+ days in advance, real-time inference doesn't make the most sense.

6. Open `CH7-02-Batch Inference` to run inference on the test set. Notice in *Figure 7.12* that we define our `model_uri` using the alias rather than the model version number.

```
import mlflow
from pyspark.sql.functions import struct, col
mlflow.set_registry_uri("databricks-uc")
model_name = "store_sales_forecasting"

logged_model = f"models:/{model_name}@Champion"

# Load model as a Spark UDF.
loaded_model = mlflow.pyfunc.spark_udf(spark, model_uri=logged_model)

df = sql("SELECT * FROM test_set")

# Predict on a Spark DataFrame.
df.withColumn('predictions', loaded_model(struct(*map(col, df.columns))))
```

Figure 7.12 – Batch forecasting on the test set using the Champion model

This inference code will always run inference on the data provided using the Champion model. If we later determine that a new version is better, we can change the model alias and run the correct model without making any code changes to the inference code.

The next practical step is to set up a workflow on a schedule. Please refer to the streaming project to see this demonstrated. This wraps up the end of this project. In *Chapter 8*, we will use the Favorita Sales data to show how easily an SQLbot can be created using the Foundational Model API.